AI INNOVATION CHALLENGE · EMPLOYEE ONBOARDING

API Contract

Request and response reference for every endpoint

This document is **generated from** `docs/openapi.json`, which is itself generated from the Zod schemas that validate requests at runtime. There is no hand-written copy of the contract anywhere, so this reference cannot drift from the running code. A CI job fails the build if the committed specification stops matching the schemas.

Field	Value
Specification	OpenAPI 3.1.0
API version	1.0.0
Endpoints	53 paths, 60 operations
Schemas	87 named components
Base URL (local)	`http://localhost:4000`
Interactive docs	`http://localhost:4000/docs` (Swagger UI)
Raw specification	`http://localhost:4000/openapi.json`
Auth	Bearer JWT. 15-minute access token, rotating refresh token.
Owner	Himanshu — schema, auth and RBAC, API contract, CI

The design rule behind every endpoint

The model proposes, deterministic code disposes, and a human authorises.
An LLM proposal is never the system output. It must survive a numbered validator chain (V1–V8) that it cannot influence, checked against a policy corpus it cannot write to, before a named person accepts responsibility for it. Several endpoints exist purely to make that chain visible: `/plans/{id}/findings`, `/plans/{id}/audit`, `/escalations` and `/clauses/{clauseId}`.

Contents

§	Section
1	Conventions: base URL, versioning, dates, pagination, async jobs
2	Authentication: the token model and why permissions are not in the JWT
3	Authorization: three roles, scopes, and the 404-not-403 rule
4	Errors: the single error envelope and every error code
5	Endpoint reference, grouped by tag
6	Worked request and response examples
7	Schema index

1 Conventions

Topic	Rule
Base URL	`http://localhost:4000` in development. All business endpoints are under `/api/v1`; operational endpoints (`/healthz`, `/readyz`, `/metrics`, `/openapi.json`, `/docs`) sit at the root.
Versioning	The major version is in the path. A breaking change means `/api/v2`; additive fields do not.
Content type	`application/json` everywhere, except document upload which is `multipart/form-data`.
Dates	Calendar dates are `YYYY-MM-DD` strings. Timestamps are RFC 3339 with an offset, e.g. `2026-11-06T09:30:00.000Z`. Never send a local time without an offset.
Casing	`camelCase` in JSON; `snake_case` in the database. The API layer translates.
Enums	`SCREAMING_SNAKE_CASE`. Treat them as closed sets: the frontend should handle an unknown value defensively, but the server rejects anything outside the enum.
Identifiers	UUID v4, except policy clauses which use stable human-meaningful ids such as `CA-ON-BEN-004`, because citations have to be readable and verifiable.
Request id	Every response carries `x-request-id`. Send your own to correlate, or let the server generate one. It appears in logs, in error bodies and in the audit trail.
Idempotency	Creates that may be retried accept an `Idempotency-Key` header.

Pagination

Every list endpoint takes `page` (default 1) and `pageSize` (default 20, maximum 100), and returns the same envelope. There are no bare arrays on list endpoints, so adding metadata later is never a breaking change.

```
GET /api/v1/employees?page=2&pageSize=20&status=ONBOARDING

200 OK
{
  "items": [ { "id": "...", "firstName": "Priya", ... } ],
  "meta": { "page": 2, "pageSize": 20, "totalItems": 47, "totalPages": 3 }
}
```

Long-running operations

Plan generation calls a language model and can take tens of seconds, so it is asynchronous. The request returns **202 Accepted** with a job id; the client polls for **named pipeline stages** rather than a meaningless spinner.

```
POST /api/v1/employees/{id}/plans          -> 202 Accepted
{ "jobId": "9b4d...", "status": "QUEUED", "pollUrl": "/api/v1/jobs/9b4d..." }

GET /api/v1/jobs/9b4d...                   -> 200 OK
{
  "jobId": "9b4d...",
  "status": "RUNNING",
  "stage": "CLAUSE_RETRIEVAL",
  "progressPercent": 45,
  "resultUrl": null,
  "error": null
}

Stages, in order:
  PROFILE_INTAKE -> JURISDICTION_RESOLUTION -> CLAUSE_RETRIEVAL
  -> PLAN_PROPOSAL -> VALIDATION -> GRAPH_AND_RISK -> PERSISTED
```

Only PLAN_PROPOSAL involves the model. Every other stage is deterministic code.

2 Authentication

The token model

Token	Lifetime	Where it lives	Purpose
Access	15 minutes	`Authorization: Bearer <token>`	Proves identity on every request
Refresh	7 days	httpOnly `eo_refresh` cookie, also returned in the body for non-browser clients	Obtains a new access token. Single use: rotated on every refresh

> **The decision worth explaining to a reviewer**
>
> The access token carries **identity and role names only** — `sub`, `email`, `eid`, `roles` and `tv`. It deliberately does **not** carry permissions or scopes.
> Permissions are resolved from the database on every request. The cost is one cached lookup; the benefit is that revoking a role takes effect on the next request instead of whenever the token happens to expire. A token proves who you are; the database decides what you may do.
> `tv` is the user's `tokenVersion`. Changing a password or forcing a logout increments the column, which kills every live access token for that user instantly — revocation with no blocklist to maintain.

Access token claims

```
{
  "sub":   "3f1a7c2e-9b4d-4f08-8d21-6b5e2a1c9d44",   // user id
  "email": "manager@demo.io",
  "eid":   "7c2e9b4d-...",                            // employee id, or null
  "roles": ["manager"],
  "tv":    0,                                         // must equal users.token_version
  "typ":   "access",                                  // refresh tokens cannot be
  "jti":   "...", "iat": 1762400000, "exp": 1762400900,
  "iss":   "employee-onboarding",
  "aud":   "employee-onboarding-api"
}
```

`typ` is checked on verify, so a refresh token cannot be presented as a bearer token.

Refresh rotation and reuse detection

Refresh tokens are stored only as a SHA-256 hash, so a database leak does not hand an attacker usable sessions. Each token belongs to a rotation *family*. Refreshing issues a new token in the same family and revokes the old one, linking it to its replacement so the chain stays auditable.

If an **already-rotated** token is presented, that means two parties hold copies and one of them is an attacker. We cannot tell which, so the entire family is revoked, a `TOKEN_REUSE_DETECTED` event is written, and both parties must sign in again.

Brute-force protection

- Rate limit of 10 attempts per 15 minutes, keyed on IP *and* email.
- After 5 consecutive failures the account locks for 15 minutes and returns **423** with `ACCOUNT_LOCKED`.
- A password comparison always runs even when the account does not exist, so response timing does not reveal which emails are registered.
- Every attempt, success or failure, is written to `auth_events`.

Development bypass

So the frontend and the six service teams can build against real routes before login is wired up, setting `DEV_AUTH_BYPASS=true` enables a header that impersonates a seeded user:

```
curl -H "x-dev-user: hr.admin@demo.io" http://localhost:4000/api/v1/employees
```

The API refuses to boot if this flag is true while `NODE_ENV` is anything other than `development`, so it cannot reach CI or a deployed environment by accident.

3 Authorization

Three roles, matching the architecture diagram

Role	Scope	Permissions	What it can and cannot do
hr_admin	**ORG**	40	Everything, across the whole organisation. **The only role that can approve a plan**, which is the single gate before any task is dispatched.
manager	**REPORTS**	14	Their own direct reports only. Completes team-setup tasks and tracks progress. Cannot approve plans. Has **no payroll grant at all**.
employee	**SELF**	11	Their own record only. Uploads documents, elects benefits, completes training. Can never read anybody else's record.

Permission and scope

A permission is `resource.action` — there are 40 of them, such as `plan.approve`, `document.verify`, `payroll.write`. Each is granted to a role **at a scope**, and the scope is what makes this real access control rather than role-name checking.

Scope	Rows the holder may touch
`SELF`	Only their own employee record
`REPORTS`	Their own record plus their direct reports
`ORG`	Every row in the tenant

Authorization runs two gates in order. First: does the caller hold this permission at all? If not, **403**. Second: is the target row inside the scope they hold it at? If not, **404**.

> **Why out-of-scope returns 404 and not 403**
>
> Returning **403 Forbidden** on another team's employee record confirms that the record exists. That is an information leak, so out-of-scope rows return **404 Not Found** — indistinguishable from a record that was never there.
> A manager holding `plan.read` at `REPORTS` passes gate one for every plan in the company, and fails gate two for all but their own reports'. The scope is evaluated against the target row in the database; it is never asserted by the token.

Where the rules live

Nothing is hard-coded in route files. The whole catalog is one source file, `packages/contracts/src/rbac.ts`, which is read by the database seed, by the runtime middleware, and by this document. Changing a grant means editing the catalog and re-running `npm run db:seed`.

```
// Guarding a route is one line. The resolver tells the middleware which
// employee the request is *about*, so scope can be checked against that row.

router.get(
  '/employees/:id',
  requirePermission('employee.read', { target: byParam('id') }),
  handler,
);

// And for list endpoints, scope becomes a SQL filter rather than an
// in-memory filter over a full table read:

const where = await employeeScopeFilter(actor, scope, 'employeeId');
```

requirePermission() in apps/api/src/middleware/authorize.ts

Verifying it works

The seed creates exactly the fixture needed to prove scoping: Priya reports to the manager account, Tom does not.

```
# as manager@demo.io
GET /api/v1/employees              -> 200, returns Priya only
GET /api/v1/employees/{priya_id}   -> 200
GET /api/v1/employees/{tom_id}     -> 404   (not 403: existence is not leaked)
PUT /api/v1/tasks/{id}/payroll     -> 403   (no payroll grant on this role at all)
```

4 Errors

Every non-2xx response in this API has the same body. There is no endpoint that returns a bare string or a differently-shaped error, so the frontend needs exactly one error handler.

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Request body failed validation",
    "fields": [
      { "path": "startDate", "message": "must be YYYY-MM-DD", "code": "invalid_string" },
      { "path": "employmentType", "message": "Invalid enum value", "code": "invalid_enum_value" }
    ],
    "requestId": "b7c1e2f0-5a3d-4c11-9f2e-8d6a4b0c1e77",
    "timestamp": "2026-11-06T09:30:00.000Z"
  }
}
```

`fields` appears only on validation failures. `requestId` is always present and matches the x-request-id response header.

Error codes

HTTP	code	Meaning and what the client should do
400	`VALIDATION_FAILED`	Body, query or params failed schema validation. Read `fields` and mark the offending inputs.
401	`UNAUTHENTICATED`	No token supplied. Redirect to sign-in.
401	`TOKEN_EXPIRED`	Access token expired. Call `/auth/refresh` once and retry.
401	`TOKEN_INVALID`	Malformed, wrong type, revoked session, or refresh reuse detected. Force a fresh sign-in; do not retry.
403	`FORBIDDEN`	Authenticated, but the role does not hold the permission. Hide the control rather than retrying.
404	`NOT_FOUND`	No such row, **or** the row is outside the caller's scope. The two are deliberately indistinguishable.
409	`CONFLICT`	State conflict, for example a draft plan already exists.
409	`APPROVAL_BLOCKED`	Guardrail V7: a LEGAL-class task is still unreviewed, so the plan cannot be approved. The blocking tasks are listed.
409	`UNSUPPORTED_JURISDICTION`	No approved policy corpus covers this location. An escalation has been raised. **This is intended behaviour, not a bug.**
413	—	Upload exceeds 10 MiB.
415	—	Upload is not PDF, JPEG or PNG.
423	`ACCOUNT_LOCKED`	Too many failed sign-ins. Locked for 15 minutes.
429	`RATE_LIMITED`	Back off and retry later.
503	`UPSTREAM_UNAVAILABLE`	The language model is unreachable. Plan generation still succeeds in `BASELINE_FALLBACK` mode.
500	`INTERNAL_ERROR`	Unexpected. Quote `requestId` when reporting.

5 Endpoint reference

All 60 operations across 53 paths, grouped by tag, exactly as published in `docs/openapi.json`. Full request and response schemas for each are browsable in Swagger UI at `/docs`.

Auth — Login, token rotation, password and invites

	Path	What it does
POST	`/api/v1/auth/accept-invite`	Activate a new hire portal account from an invite token
POST	`/api/v1/auth/change-password`	Change password and invalidate all existing sessions
POST	`/api/v1/auth/login`	Exchange credentials for an access and refresh token pair
POST	`/api/v1/auth/logout`	Revoke the current refresh token, or every token for the user
GET	`/api/v1/auth/me`	The caller's identity, roles and effective permissions
POST	`/api/v1/auth/refresh`	Rotate the refresh token and issue a new access token

Employees — New-hire records

	Path	What it does
GET	`/api/v1/employees`	List employees visible to the caller
POST	`/api/v1/employees`	Create a new-hire record
GET	`/api/v1/employees/{id}`	Fetch one employee
PATCH	`/api/v1/employees/{id}`	Update an employee
POST	`/api/v1/employees/{id}/invite`	Invite a new hire to the employee portal

Plans — AI plan generation, graph, risk and approval

	Path	What it does
POST	`/api/v1/employees/{id}/plans`	Request AI generation of an onboarding plan
GET	`/api/v1/jobs/{id}`	Poll an async job
GET	`/api/v1/plans`	List plans visible to the caller
GET	`/api/v1/plans/{id}`	A plan with its tasks, citations and confidence breakdown
POST	`/api/v1/plans/{id}/approval`	Approve a plan — the only path to task dispatch
GET	`/api/v1/plans/{id}/graph`	Dependency DAG, topological order and critical path
GET	`/api/v1/plans/{id}/risks`	Tasks that will miss day one, and why

Tasks — The cross-plan work queue

	Path	What it does
PATCH	`/api/v1/plans/{id}/tasks/{taskId}`	Edit a task
POST	`/api/v1/plans/{id}/tasks/{taskId}/escalations`	Escalate a task to compliance
GET	`/api/v1/tasks`	Cross-plan task queue for the caller

Service: Documents — Document collection and verification

	Path	What it does
GET	/api/v1/documents	List document requests in scope
POST	/api/v1/tasks/{taskId}/document	Upload a document against a task
GET	/api/v1/tasks/{taskId}/document-request	Document request detail for a task
POST	/api/v1/tasks/{taskId}/document/verification	Verify or reject an uploaded document

Service: Equipment — IT asset provisioning

	Path	What it does
GET	/api/v1/equipment	IT work queue
GET	/api/v1/equipment/catalog	Equipment catalog with lead times
PATCH	/api/v1/tasks/{taskId}/equipment	Update equipment status, serial or delivery date
POST	/api/v1/tasks/{taskId}/equipment	Order equipment for a task

Service: Payroll — Payroll setup (pay-affecting)

	Path	What it does
GET	/api/v1/tasks/{taskId}/payroll	Payroll setup for a task
PUT	/api/v1/tasks/{taskId}/payroll	Create or update payroll setup
POST	/api/v1/tasks/{taskId}/payroll/verification	Verify payroll setup before the first run

Service: Benefits — Benefit enrolment windows

	Path	What it does
GET	/api/v1/benefits/enrollments	Benefits work queue
GET	/api/v1/benefits/plans	Available benefit plans
GET	/api/v1/tasks/{taskId}/benefit-enrollment	Benefit enrolment for a task, with the window countdown
POST	/api/v1/tasks/{taskId}/benefit-enrollment	Elect or waive a benefit

Service: Training — Mandatory and role training

	Path	What it does
POST	/api/v1/tasks/{taskId}/training	Assign a course to a task
POST	/api/v1/tasks/{taskId}/training/completion	Record training completion
GET	/api/v1/training/assignments	L&D work queue
GET	/api/v1/training/courses	Course catalog

Service: Manager — Team setup tasks

	Path	What it does
GET	/api/v1/tasks/{taskId}/manager-task	Manager setup task detail
PUT	/api/v1/tasks/{taskId}/manager-task	Set buddy, day-one agenda or notes

Policies — The approved knowledge corpus and citation resolution

	Path	What it does
GET	/api/v1/clauses/{clauseId}	Resolve a citation to its source clause
GET	/api/v1/policies	Browse the approved policy corpus
GET	/api/v1/policies/coverage	Which jurisdictions we hold an approved corpus for

Harness — Validator findings, escalations, audit and evaluation evidence

	Path	What it does
GET	/api/v1/escalations	Open escalations — what the system refused to decide alone
POST	/api/v1/escalations/{id}/resolution	Resolve or dismiss an escalation
GET	/api/v1/evals/latest	Most recent evaluation run against the golden set
GET	/api/v1/harness/stats	Live harness counters for the evidence dashboard
GET	/api/v1/plans/{id}/audit	Append-only audit trail for a plan
GET	/api/v1/plans/{id}/findings	What the validator chain caught

Admin — Users, roles and permissions

	Path	What it does
GET	/api/v1/admin/roles	Roles with their permissions and scopes
GET	/api/v1/admin/users	List user accounts
POST	/api/v1/admin/users	Create a user account
PATCH	/api/v1/admin/users/{id}	Suspend, unlock or force-logout a user
PUT	/api/v1/admin/users/{id}/roles	Grant or revoke roles

Reference — Jurisdictions and departments

	Path	What it does
GET	/api/v1/departments	Departments
GET	/api/v1/jurisdictions	Supported jurisdictions and their business-day calendars

Ops — Health, readiness and metrics

	Path	What it does
GET	/healthz	Liveness
GET	/readyz	Readiness — database and LLM reported separately

6 Worked request and response examples

6.1 Sign in

```
POST /api/v1/auth/login
Content-Type: application/json

{ "email": "manager@demo.io", "password": "Onboard!ng2026" }

200 OK
Set-Cookie: eo_refresh=...; HttpOnly; SameSite=Strict; Path=/api/v1/auth
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": "3f1a7c2e-...",
    "email": "manager@demo.io",
    "status": "ACTIVE",
    "mustChangePassword": false,
    "employeeId": "7c2e9b4d-...",
    "firstName": "David",
    "lastName": "Chen",
    "roles": ["manager"],
    "permissions": [
      { "key": "employee.read",     "resource": "employee", "action": "read",  "scope": "REPORTS" },
      { "key": "managertask.write", "resource": "managertask","action": "write","scope": "REPORTS" }
    ]
  }
}
```

The frontend uses `permissions` to decide which controls to render. The server never trusts that decision and re-checks on every request.

6.2 Create a new hire

```
POST /api/v1/employees
Authorization: Bearer <hr_admin token>

{
  "firstName": "Priya",
  "lastName": "Nair",
  "personalEmail": "priya@example.com",
  "jobTitle": "Software Engineer",
  "roleFamily": "engineering",
  "jurisdictionCode": "CA-ON",
  "employmentType": "FULL_TIME",
  "workMode": "HYBRID",
  "startDate": "2026-12-01",
  "unionMember": false,
  "notes": "Relocating from Bengaluru."
}

201 Created
{ "id": "...", "employeeNumber": "EMP-0003", "status": "PRE_HIRE", ... }
```

`notes` is stored as UNTRUSTED text. It reaches the prompt inside a delimited data block and is never followed as instruction.

6.3 A plan with a grounded citation

```
GET /api/v1/plans/{id}

200 OK
{
  "id": "...",
  "employeeName": "Priya Nair",
  "jurisdictionCode": "CA-ON",
  "status": "AWAITING_APPROVAL",
  "generationMode": "LLM_GROUNDED",
  "confidence": 0.86,
  "confidenceBreakdown": {
    "citationQuoteMatch": 0.97, "retrievalScore": 0.81,
    "templateCorroboration": 0.90, "validatorCleanRate": 0.93,
    "schemaFirstPass": 1.0
  },
  "refusalReason": null,
  "degradedReason": null,
  "tasks": [
    {
      "id": "...",
      "taskType": "PROVISION_LAPTOP",
      "service": "EQUIPMENT",
      "title": "Order and configure laptop",
      "ownerRole": "IT",
      "dueDate": "2026-11-24",
      "earliestStartDate": "2026-11-17",
      "authorityClass": "ROUTINE",
      "origin": "LLM_ACCEPTED",
      "reviewState": "OK",
      "status": "NOT_STARTED",
      "isAtRisk": true,
      "riskReason": "Lead time is 5 business days; only 3 remain before the start date.",
      "onCriticalPath": true,
      "dependsOn": [],
      "citations": [
        {
          "clauseId": "CA-ON-EQP-002",
          "documentTitle": "Ontario Onboarding Policy",
          "isMock": true,
          "quotedText": "Equipment must be available to the worker on their first day.",
          "quoteMatch": 0.97,
          "retrievalRank": 1
        }
      ]
    }
  ]
}
```

`confidence` is computed in code from five measured signals, never self-reported by the model. `dueDate` is computed by the deadline engine, never taken from the model.

6.4 Refusal — the behaviour to demo

Generating a plan for the Singapore contractor, for whom we hold no approved policy corpus. The system does not invent a plausible plan:

```
GET /api/v1/plans/{id}

200 OK
{
  "status": "REFUSED",
  "generationMode": "BASELINE_FALLBACK",
  "refusalReason": "No approved policy source covers CONTRACTOR onboarding in SG.",
  "confidence": null,
  "tasks": []
}

GET /api/v1/escalations
{
  "items": [{
    "reasonCode": "UNSUPPORTED_JURISDICTION",
    "detail": "Plan generation refused: no policy clauses indexed for SG.",
    "assignedRole": "COMPLIANCE",
    "status": "OPEN"
  }]
}
```

6.5 Approval blocked by guardrail V7

```
POST /api/v1/plans/{id}/approval
{ "decision": "APPROVE", "acknowledgement": true }

409 Conflict
{
  "error": {
    "code": "APPROVAL_BLOCKED",
    "message": "2 legally-required tasks are still unreviewed.",
    "fields": [
      { "path": "tasks[0]", "message": "RIGHT_TO_WORK_CHECK is NEEDS_REVIEW (LEGAL)" },
      { "path": "tasks[1]", "message": "COLLECT_TAX_DECLARATION is NEEDS_REVIEW (PAY_AFFECTING)" }
    ],
    "requestId": "...", "timestamp": "..."
  }
}
```

The system will not let a human rubber-stamp past a compliance item. `acknowledgement: true` is required, so approval is always an explicit act.

6.6 What the harness caught

```
GET /api/v1/plans/{id}/findings

200 OK
{
  "items": [
    {
      "validator": "V2_CITATION_INTEGRITY", "severity": "REJECT",
      "code": "CLAUSE_NOT_FOUND", "action": "REJECTED",
      "message": "Task cited CA-ON-BEN-999, which does not exist. Task dropped.",
      "taskTitle": "Enrol in extended dental cover"
    },
    {
      "validator": "V4_MANDATORY_COVERAGE", "severity": "WARN",
      "code": "MANDATORY_TASK_MISSING", "action": "INSERTED",
      "message": "Model omitted HEALTH_AND_SAFETY_TRAINING, required in CA-ON. Inserted.",
      "taskTitle": "Ontario Occupational Health & Safety Awareness"
    },
    {
      "validator": "V6_DEADLINE_RECOMPUTE", "severity": "INFO",
      "code": "DEADLINE_DIVERGED", "action": "REPAIRED",
      "message": "Model proposed 2026-12-05; engine computed 2026-12-02. Code wins.",
      "taskTitle": "Collect tax declaration"
    }
  ],
  "meta": { "page": 1, "pageSize": 20, "totalItems": 3, "totalPages": 1 }
}
```

This endpoint is the evidence panel. Each row is one validator decision, persisted at generation time.

7 Schema index

All 87 named schemas in the specification. Each is generated from a Zod schema that also validates the corresponding request or response at runtime, so these names are directly importable as TypeScript types from `@eo/contracts`.

AcceptInviteRequest	EquipmentAssignment	PlanListResponse
AdminUser	EquipmentAssignmentListResponse	PlanRisk
AdminUserListResponse	EquipmentCatalogItem	PlanRisksResponse
ApprovePlanRequest	ErrorResponse	PlanSummary
ApprovePlanResponse	Escalation	PlanTask
AssignRolesRequest	EscalationListResponse	PolicyClause
AuditEvent	EvalMetrics	PolicyCoverage
AuditListResponse	FieldError	PolicyDocument
AuthUser	FindingListResponse	PolicyDocumentListResponse
BenefitEnrollment	GeneratePlanRequest	Readiness
BenefitEnrollmentListResponse	HarnessStats	RefreshRequest
BenefitPlan	Health	RefreshResponse
ChangePasswordRequest	InviteEmployeeRequest	ResolveClauseResponse
CompleteTrainingRequest	InviteEmployeeResponse	ResolveEscalationRequest
ConfidenceBreakdown	JobAccepted	Role
CreateEmployeeRequest	JobState	TaskCitation
CreateEquipmentAssignmentRequest	Jurisdiction	TaskListResponse
CreateEscalationRequest	LoginRequest	TrainingAssignment
CreateTrainingAssignmentRequest	LoginResponse	TrainingAssignmentListResponse
CreateUserRequest	LogoutRequest	TrainingCourse
Department	ManagerTask	UpdateEmployeeRequest
Document	MeResponse	UpdateEquipmentAssignmentRequest
DocumentRequest	MessageResponse	UpdatePlanTaskRequest
DocumentRequestListResponse	PageMeta	UpdateUserRequest
EffectivePermission	PayrollSetup	UpsertManagerTaskRequest
ElectBenefitRequest	Plan	UpsertPayrollSetupRequest
Employee	PlanGraph	ValidationFinding
EmployeeListResponse	PlanGraphEdge	VerifyDocumentRequest
EmployeeSummary	PlanGraphNode	VerifyPayrollRequest

Using these types in the frontend

Do not redeclare these shapes in React. Import them:
`import type { Plan, PlanTask, AuthUser } from '@eo/contracts';`
The package is a workspace dependency, so the types the API validates against and the types the UI compiles against are the same objects. If the contract changes, the frontend fails to compile rather than failing at runtime in the demo.